

Univerza v Mariboru
Fakulteta za elektrotehniko, računalništvo in informatiko

KONČNI PROJEKT

Triatlon — informacijski sistem za
spremljanje triatlonskih rezultatov

Dokumentacija projekta · navodila za namestitev · navodila za uporabo

Ime in priimek: Aleks Maršnak
Vpisna številka: 1002681669
Predmet: Orodja za razvoj aplikacij
Študijsko leto: 2025/2026

Kazalo vsebine

Kazalo vsebine.....	2
1. Uvod.....	3
2. Arhitektura sistema	3
2.1 Pregled komponent in tehnologij	3
2.2 Uporabljene knjižnice (NuGet).....	4
3. Podatkovna baza	4
3.1 Entitetno-relacijski diagram	4
3.2 Opis relacij.....	5
4. Opis posameznih komponent	5
4.1 Naloga 1 — Konzolna aplikacija za uvoz	5
4.2 Naloga 2 — REST API.....	5
4.3 Naloga 3 — Namizna aplikacija (WPF)	6
4.4 Naloga 4 — Spletna aplikacija (ASP.NET Core Razor Pages)	6
5. Navodila za namestitev	7
5.1 Sistemske zahteve	7
5.2 Korak 1 — Priprava podatkovne baze.....	7
5.3 Korak 2 — Nastavitev povezave do baze	8
5.4 Korak 3 — Odpiranje rešitve in obnova paketov	8
5.5 Korak 4 — Polnjenje baze s podatki.....	9
5.6 Korak 5 — Zagon strežnika in aplikacij	9
6. Navodila za uporabo	9
6.1 REST API in Swagger.....	9
6.2 Namizna aplikacija (administrator).....	9
6.3 Spletna aplikacija (navadni uporabnik)	10
6.4 Pogoste težave in rešitve.....	10
7. Zaključek	10

1. Uvod

Ta dokument predstavlja zaključno dokumentacijo končnega projekta pri predmetu Orodja za razvoj aplikacij. Projekt je nastal postopno skozi štiri sprotne računalniške vaje (Naloge 1–4) in skupaj tvori celovit informacijski sistem za spremljanje triatlonskih rezultatov.

Naročnik sistema je **Bruno**, strasten triatlonec, ki si želi enostaven dostop do svojih in tujih rezultatov brez nepotrebnega iskanja po spletnih straneh, prepolnih oglasov. Sistem mora omogočati uvoz preteklih rezultatov, urejanje podatkov, prikaz rezultatov in statistike ter dostop do podatkov preko aplikacijskega vmesnika (API).

Sistem je sestavljen iz štirih ločenih programov, ki si delijo skupno podatkovno bazo PostgreSQL in komunicirajo preko skupne REST storitve:

- **Konzolna aplikacija za uvoz (Naloga 1)** — prebere CSV datoteke z rezultati in jih napolni v podatkovno bazo.
- **REST API (Naloga 2)** — osrednja poslovna logika; bere in piše v bazo ter izpostavlja podatke vsem odjemalcem.
- **Namizna aplikacija WPF (Naloga 3)** — administratorski vmesnik za upravljanje (CRUD) tekmovalcev, tekmovanj, rezultatov in uporabnikov.
- **Spletna aplikacija ASP.NET Core (Naloga 4)** — vmesnik za navadne uporabnike (iskanje, lestvice, statistika, primerjave).

Vse štiri komponente so zbrane v eni Visual Studio rešitvi (solution) z imenom Triatlon.sln. Poslovna logika je dosledno ločena od uporabniških vmesnikov in se v celoti nahaja v REST storitvi — namizna in spletna aplikacija sami ne dostopata neposredno do baze, temveč vedno preko API-ja.

2. Arhitektura sistema

Sistem sledi večplastni arhitekturi, v kateri je podatkovna baza skupno hranišče, REST API edina točka dostopa do poslovne logike, oba uporabniška vmesnika (namizni in spletni) pa sta zgolj odjemalca te storitve.

Tok podatkov v sistemu poteka takole: konzolna aplikacija enkratno napolni bazo iz CSV datotek; REST API nato bere in zapisuje podatke v bazo; namizna in spletna aplikacija pošiljata HTTP zahteve API-ju ter prikazujeta vrnjene podatke uporabniku.

2.1 Pregled komponent in tehnologij

Komponenta	Tehnologija	Ogrodje / ciljni okvir	Vloga
Naloga 1 — Uvoz	C# (konzolna)	.NET 8	Polnjenje baze iz CSV
Naloga 2 — REST API	ASP.NET Core Web API	.NET 8	Poslovna logika, dostop do baze

Komponenta	Tehnologija	Ogrodje / ciljni okvir	Vloga
Naloga 3 — Namizna	WPF (MVVM)	.NET 9 (Windows)	Admin vmesnik (CRUD)
Naloga 4 — Spletna	ASP.NET Core Razor Pages	.NET 9	Uporabniški vmesnik
Baza	PostgreSQL	v14 ali novejša	Skupno hranišče podatkov

Tabela 1: Komponente sistema in uporabljene tehnologije

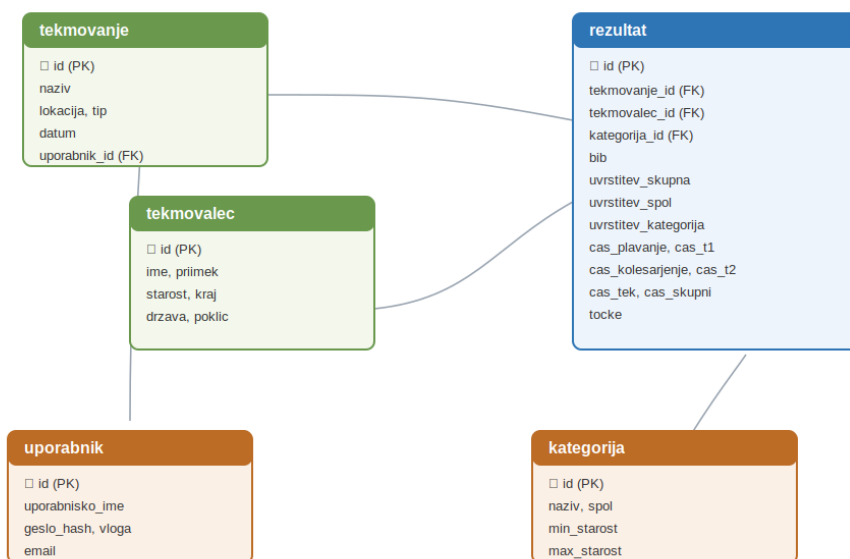
2.2 Uporabljene knjižnice (NuGet)

- **Npgsql** — gonilnik za povezavo z bazo PostgreSQL (uporabljen v konzolni aplikaciji in API-ju).
- **CsvHelper** — branje in razčlenjevanje CSV datotek pri uvozu.
- **Swashbuckle.AspNetCore (Swagger)** — samodejna dokumentacija in testno okolje za REST API.
- **HttpClient + System.Text.Json** — komunikacija odjemalcev z API-jem (vgrajeno v .NET).
- **Bootstrap 5, Chart.js, Font Awesome** — videz in grafi v spletni aplikaciji.

3. Podatkovna baza

Podatkovni model je sestavljen iz petih medsebojno povezanih tabel. Osrednja tabela je rezultat, ki preko tujih ključev povezuje tekmovalca, tekmovanje in kategorijo. Tabela uporabnik hrani račune (administratorje in navadne uporabnike).

3.1 Entitetno-relacijski diagram



Slika 1: ER diagram podatkovne baze (5 tabel)

3.2 Opis relacij

- Eno tekmovanje ima več rezultatov (1 : N).
- En tekmovalec ima več rezultatov, po enega za vsako tekmovanje, ki se ga je udeležil (1 : N).
- Ena kategorija (npr. M40, F30) združuje več rezultatov (1 : N).
- En uporabnik (administrator) je lahko vnesel več tekmovanj (1 : N).

4. Opis posameznih komponent

4.1 Naloga 1 — Konzolna aplikacija za uvoz

Konzolna aplikacija Naloga1_Uvoz prebere izvirne CSV datoteke z rezultati triatlonskih tekmovanj in jih napolni v podatkovno bazo. Datoteke so razvrščene v tri mape glede na tip tekmovanja: IRONMAN, IRONMAN 70.3 in Ultra-triatlon.

Pri uvozu aplikacija razčleni vsako vrstico CSV, manjkajoče vrednosti nadomesti z oznako EMPTY, iz imena datoteke razbere lokacijo in leto tekmovanja ter podatke vstavi v ustrezne tabele (tekmovanje, tekmovalec, kategorija, rezultat). Ob koncu izpiše statistiko uvoza (število tekmovanj, rezultatov in porabljen čas).

4.2 Naloga 2 — REST API

REST API je srce sistema. Vsebuje celotno poslovno logiko in je edina komponenta, ki neposredno komunicira z bazo. Zgrajen je po vzorcu ASP.NET Core Web API s krmilniki (controllers), ki za vsako entiteto izpostavljajo CRUD operacije ter dodatne poizvedbe (iskanje, lestvica, statistika, filtriranje, primerjava).

API ima vključen Swagger, ki omogoča pregled vseh končnih točk in njihovo testiranje neposredno v brskalniku. CORS je nastavljen tako, da dovoljuje zahteve iz namizne in spletne aplikacije.

Glavne skupine končnih točk (vsaka podpira ustrezne metode HTTP):

Krmilnik	Osnovna pot	Podprte metode
Tekmovalci	/api/Tekmovalci	GET, GET/{id}, GET iskanje, POST, PUT, DELETE
Tekovanja	/api/Tekovanja	GET, GET/{id}, GET tip/{tip}, POST, PUT, DELETE
Rezultati	/api/Rezultati	GET, lestvica, statistika, filter, primerjava, izvoz, POST, PUT, DELETE
Kategorije	/api/Kategorije	GET, GET/{id}
Uporabniki	/api/Uporabniki	GET, GET/{id}, POST, PUT, DELETE

Tabela 2: Pregled končnih točk REST API

4.3 Naloga 3 — Namizna aplikacija (WPF)

Namizna aplikacija Naloga3_WPF je namenjena administratorjem. Zgrajena je po vzorcu MVVM (Model-View-ViewModel), ki ločuje uporabniški vmesnik (XAML) od poslovne logike (ViewModel). Vsa komunikacija z bazo poteka preko razreda ApiService, ki kliče REST API.

Aplikacija je razdeljena na pet zavihkov:

- **Tekmovalci** — pregled, iskanje, filtriranje po državi, dodajanje, urejanje, brisanje in izvoz v CSV.
- **Tekmovanja** — upravljanje tekmovanj z validacijo in potrditvenim dialogom ob brisanju.
- **Rezultati** — prikaz rezultatov in izvoz v CSV.
- **Statistika** — pregled skupnih števil in razčlenitev po tipu tekmovanja.
- **Uporabniki** — upravljanje uporabniških računov in njihovih vlog.

Aplikacija podpira dva jezika (slovenščino in angleščino) s preklopom v zgornjem desnem kotu ter validira vnos pred shranjevanjem (npr. ime in priimek tekmovalca sta obvezna).

4.4 Naloga 4 — Spletna aplikacija (ASP.NET Core Razor Pages)

Spletna aplikacija Naloga4_Splet je namenjena navadnim uporabnikom in je dostopna v brskalniku. Zgrajena je z Razor Pages (ločitev .cshtml in code-behind .cshtml.cs); vsa komunikacija z API-jem poteka preko skupnega razreda ApiService.

Glavne funkcionalnosti:

- Domača stran s statistiko, zadnjimi tekmovanji in tortnim grafom tipov.
- Iskanje tekmovalcev po imenu ali priimku.
- Profil tekmovalca z grafom napredka skozi čas (Chart.js).
- Seznam tekmovanj s filtrom po lokaciji in tipu.

- Lestvica za izbrano tekmovanje z medaljami za prve tri.
- Filtriranje rezultatov po državi, kategoriji in starosti.
- Primerjava dveh tekmovalcev z določitvijo boljšega.
- Registracija in prijava uporabnika (seja).

Aplikacija uporablja Bootstrap 5 za odziven videz in teče vzporedno z REST API-jem.

5. Navodila za namestitev

To poglavje opisuje, kako od začetka postaviti celoten sistem na nov računalnik. Koraki si sledijo v navedenem vrstnem redu.

5.1 Sistemske zahteve

- **Operacijski sistem:** Windows 10/11 (namizna aplikacija WPF deluje le na sistemu Windows).
- **.NET SDK 8.0 in 9.0** — API in uvoz ciljata .NET 8, namizna in spletna aplikacija .NET 9. Prenos: <https://dotnet.microsoft.com/download>.
- **PostgreSQL 14** ali novejši, vključno z orodjem pgAdmin. Prenos: <https://www.postgresql.org/download/>.
- **Visual Studio 2022** z delovnima obremenitvama »ASP.NET and web development« ter ».NET desktop development«.
- **Izvorni podatki** (CSV datoteke z rezultati) v mapi Race-Results, ki je priložena v arhivu.

5.2 Korak 1 — Priprava podatkovne baze

1. Zaženite pgAdmin in se povežite na lokalni strežnik PostgreSQL.
2. Ustvarite novo bazo z imenom triathlon.

Pomembno: aplikacije ne ustvarjajo tabel samodejno (konzolni uvoz uporablja le ukaze INSERT). Zato je treba tabele ustvariti enkratno z naslednjo SQL skripto. V pgAdmin odprite »Query Tool« nad bazo triathlon in zaženite spodnji SQL.

```
CREATE TABLE uporabnik (  
    id SERIAL PRIMARY KEY,  
    uporabnisko_ime TEXT,  
    geslo_hash TEXT,  
    vloga TEXT,  
    email TEXT  
);
```

```
CREATE TABLE kategorija (  
    id SERIAL PRIMARY KEY,  
    naziv TEXT,  
    spol TEXT,  
    min_starost INTEGER,  
    max_starost INTEGER  
);
```

```
CREATE TABLE tekmovanje (  
    id SERIAL PRIMARY KEY,  
    naziv TEXT,  
    spol TEXT,  
    min_starost INTEGER,  
    max_starost INTEGER  
);
```

```
id SERIAL PRIMARY KEY,
naziv TEXT,
lokacija TEXT,
tip TEXT,
datum DATE,
uporabnik_id INTEGER REFERENCES uporabnik(id)
);

CREATE TABLE tekmovalec (
id SERIAL PRIMARY KEY,
ime TEXT,
priimek TEXT,
starost INTEGER,
kraj TEXT,
drzava TEXT,
poklic TEXT
);

CREATE TABLE rezultat (
id SERIAL PRIMARY KEY,
tekmovanje_id INTEGER REFERENCES tekmovanje(id),
tekmovalec_id INTEGER REFERENCES tekmovalec(id),
kategorija_id INTEGER REFERENCES kategorija(id),
bib TEXT,
uvrstitev_skupna INTEGER,
uvrstitev_spol INTEGER,
uvrstitev_kategorija INTEGER,
cas_plavanje TIME,
cas_t1 TIME,
cas_kolesarjenje TIME,
cas_t2 TIME,
cas_tek TIME,
cas_skupni TIME,
tocke DOUBLE PRECISION
);
```

5.3 Korak 2 — Nastavitev povezave do baze

Povezovalni niz (connection string) je treba prilagoditi svojim podatkom (uporabniško ime in geslo PostgreSQL). Nastaviti ga je treba na dveh mestih:

- **REST API:** datoteka `Triatlon/appsettings.json`, ključ `ConnectionStrings` → `DefaultConnection`.
- **Konzolni uvoz:** datoteka `Naloga1_Uvoz/UvozPodatkov.cs`, spremenljivka `ConnString` v razredu `BazaPodatkov`.

Primer povezovalnega niza:

```
Host=127.0.0.1;Port=5432;Database=triathlon;Username=postgres;Password=VASE_GESLO
```

5.4 Korak 3 — Odpiranje rešitve in obnova paketov

3. V Visual Studio 2022 odprite datoteko `Triatlon.sln`.
4. Visual Studio ob odpiranju samodejno obnovi vse NuGet pakete. Po želji ročno: desni klik na rešitev → »Restore NuGet Packages«.
5. Prevedite rešitev (Build → Build Solution), da preverite, da ni napak.

Opomba: solution se sklicuje na projekt Naloga4_Splet (spletna aplikacija). Mapa Naloga4_Splet mora biti v arhivu poleg mape Triatlon, sicer Visual Studio projekta ne najde.

5.5 Korak 4 — Polnjenje baze s podatki

6. V datoteki UvozPodatkov.cs preverite spremenljivko korenMapa, da kaže na mapo Race-Results/Race-Results na vašem računalniku.
7. Kot zagonski projekt nastavite Naloga1_Uvoz (desni klik → »Set as Startup Project«) in ga zaženite.
8. Aplikacija prebere CSV datoteke in napolni bazo. Ob koncu izpiše statistiko (število tekmovalcev in rezultatov ter čas uvoza). Ta korak izvedete samo enkrat.

5.6 Korak 5 — Zagon strežnika in aplikacij

Za pravilno delovanje mora vedno najprej teči REST API, šele nato namizna ali spletna aplikacija.

9. Zaženite projekt Triatlon (REST API). Privzeto teče na naslovu <https://localhost:7144>, odpre se Swagger.
10. Za administracijo zaženite Naloga3_WPF (namizna aplikacija).
11. Za uporabniški del zaženite Naloga4_Splet (spletna aplikacija) in jo odprite v brskalniku.

Hkraten zagon: v Visual Studio lahko prek »Configure Startup Projects« → »Multiple startup projects« nastavite, da se API in izbrani odjemalec zažene sočasno. Odjemalci pričakujejo API na naslovu <https://localhost:7144>; če port spremenite, ga posodobite v razredu ApiService.

6. Navodila za uporabo

6.1 REST API in Swagger

Ob zagonu API-ja se v brskalniku odpre Swagger na naslovu <https://localhost:7144/swagger>. Tu so našteje vse končne točke. S klikom na posamezno končno točko in gumbom »Try it out« lahko zahtevo izvedete neposredno (npr. GET /api/Tekmovalci vrne seznam tekmovalcev). To je uporabno za testiranje in pregled podatkov brez uporabniškega vmesnika.

6.2 Namizna aplikacija (administrator)

Aplikacija se odpre v oknu s petimi zavihki. Tipičen potek dela:

12. Izberite zavihek (npr. Tekmovalci).
13. Za iskanje vpišite ime ali priimek v iskalno polje; za filtriranje uporabite spustni seznam držav.
14. Za urejanje kliknite vrstico v tabeli — podatki se izpišejo v obrazec na desni.
15. Za dodajanje kliknite »Nov«, izpolnite obrazec in kliknite »Shrani«.
16. Za brisanje izberite vrstico in kliknite »Izbriši« ter potrdite dialog.
17. Za izvoz seznama v CSV uporabite gumb »Izvozi CSV«.
18. Jezik vmesnika preklopite z gumbom (ENG/SLO) v zgornjem desnem kotu.

Če je vnos neveljaven (npr. manjka obvezno ime), aplikacija v statusni vrstici spodaj izpiše opozorilo in podatkov ne shrani.

6.3 Spletna aplikacija (navadni uporabnik)

Spletno aplikacijo odprite v brskalniku. Osnovni načini uporabe:

19. Na domači strani uporabite iskalnik ali hitre povezave do funkcionalnosti.
20. V zavihku Tekmovalci poiščite tekmovalca in odprite njegov profil za podroben pregled rezultatov in graf napredka.
21. V zavihku Tekmovanja filtrirajte po lokaciji ali tipu in odprite lestvico izbranega tekmovanja.
22. Z orodjem Filter omejite rezultate po državi, kategoriji ali starosti.
23. V Primerjavi izberite dva tekmovalca za vzporedni prikaz statistik.
24. Za shranjevanje seje se registrirajte in prijavite; po prijavi je vaše ime vidno v navigaciji.

6.4 Pogoste težave in rešitve

Težava	Vzrok in rešitev
Aplikacija ne prikaže podatkov	REST API ne teče. Najprej zaženite projekt Triatlon, šele nato odjemalca.
Napaka »Connection string ni nastavljen« ali napaka povezave	Napačno geslo ali ime baze. Preverite appsettings.json (API) in UvozPodatkov.cs (uvoz).
Uvoz pade z napako o neobstoječi tabeli	Tabele niso bile ustvarjene. Najprej zaženite SQL skripto iz poglavja 5.2.
Mapa s podatki ne obstaja	Spremenljivka korenMapa v UvozPodatkov.cs kaže na napačno pot. Popravite jo na svojo lokacijo mape Race-Results.
Visual Studio ne najde projekta Naloga4_Splet	Mapa Naloga4_Splet mora biti poleg mape Triatlon, kakor določa pot v Triatlon.sln.

Tabela 3: Odpravljanje pogostih težav

7. Zaključek

V okviru končnega projekta je bil izdelan celovit informacijski sistem za spremljanje triatlonskih rezultatov, sestavljen iz štirih medsebojno povezanih komponent: konzolne aplikacije za uvoz, REST storitve s poslovno logiko, namizne administratorske aplikacije in spletne aplikacije za navadne uporabnike.

Sistem dosledno ločuje poslovno logiko od uporabniških vmesnikov — vsa logika je zbrana v REST storitvi, oba odjemalca pa do podatkov dostopata izključno preko API-ja. S tem so izpolnjene zahteve naročnika Bruna: uvoz preteklih rezultatov, urejanje podatkov, prikaz rezultatov in statistike na spletu ter dostop do podatkov preko aplikacijskega vmesnika.

Celotna rešitev je zbrana v eni Visual Studio rešitvi in jo je mogoče postaviti po korakih, opisanih v tem dokumentu. Izvorna koda vseh nalog je priložena v ločenem arhivu ZIP.